AI Assignment 3: Handwriting Recognition

資工二乙：張惟喆

學號：412262371

1 Introduction

這個報告的主要的目的是實作並且比較三種不同的手寫辨識模型，包含 KNN, Decision Tree, Linear Classifier。使用的是 MINST 的訓練資料，在接下來的內容中，將會針對實作的方法、如何改進、Operators、Performance 等部分進行探討。

2 Learning and Predict approaches

2.1 KNN

在訓練階段，KNN 沒有實質上的訓練，他僅僅是將訓練資料儲存起來，然後就算訓練完成了，所以也被說是 lazy learning。

在預測階段，他會計算新的輸入中的所有特徵和所有訓練樣本的歐氏距離，然後選出前 k 個歐氏距離最小的，用多數決票選出這 k 個中出現最多次的 label 作為最後的預測。

2.2 Linear Classifier

在訓練階段，Linear Classifier 會先創建 10 個 Binary Classifier，給予每一個 features 一個 weight，接著透過訓練資料嘗試預測，如果預測錯了，就修改這個 weight 使其能更接近正確的結果，也就是希望每個 binary classifier 都能正確的區分所有資料成「某數」和「非某數」兩類。

在預測階段，會把給定的圖片輪流對 10 個 classifier 做內積，內積的結果越大，表示這張圖片和這個 classifier 更契合。所以經過 10 個 classifier 的內積之後，選出內積結果最高的 classifier 作為最後的預測。

2.3 Decision Tree

在訓練階段，Decision Tree 會嘗試所有 features 和所有 threshold 的組合，找出 Gini impurity 最低的組合，並在這邊切割資料成左邊和右邊，直到所有葉節點都代表某一個類別。萬一到達設定的深度上限，但仍然有超過一種的數字被分在同一類，這時候就會用 `majority_class()`，靠多數決票選出這個葉節點代表的預測數字。

在預測階段，對於每一張圖片，都從根結點開始，根據我們剛剛建立好的 Tree，依序在不同 features 上根據 threshold，決定要往左節點還是右節點，一路到達某個葉節點，這個葉節點就是模型對於該圖片的預測結果。

3 Optimizations

3.1 KNN

Learning Approach	Average label vector
Predict Approach	對所有 label vector 的歐氏距離

原本的 KNN 在預測的階段會比較完所有輸入和訓練資料(60000 筆)的距離，這樣訓練資料和測試資料加起來總共(10000 (test data) + 60000 (train data)) * 28 * 28 (pixels) * 60000 (train data) = 3.2 Trillion 的計算量，非常龐大而且算不完。

所以改進版本在訓練時期不再是單單把訓練資料儲存起來而已。而是根據不同的 label，將所有同種 label 的 image 相加後取平均，這樣會得到 10 個數字的平均圖像。

預測時只需要比較輸入和這 10 個平均圖像，找出與輸入距離最短的圖像，該圖像代表的數字就作為最後的預測，雖然這麼做會使得圖片變得較容易誤判，但是得到的是更快的預測時間，我覺得是不錯的 trade off。

3.2 Linear Classifier

Learning Approach	調整 10 個 binary classifier weight
Max epochs	250
Learning rate	1.0
Predict Approach	輪流對 10 個 binary classifier 做內積

原本的 Linear Classifier 每次都會重新訓練，而且每次訓練都有非常高的 epoch 上限，所以訓練時間非常久。

所以在改進版本中，逐步測試最後把 max epoch 設定在 250，並且一旦訓練完模型，會自動儲存這些權重，新增了 save_weights()和 load_weights()兩個函式，這樣下次如果還用相同的 max epoch 執行程式，我們就可以直接取用模型，略過耗時的訓練部分。

3.3 Decision Tree

Learning Approach	嘗試所有 feature 和 value 來找到分割點
Max depth	10
Feature skipping	每四個 pixel 取一次
Predict Approach	根據 decision tree 上的 threshold 來決定要往左子樹還是右子樹

原本的 Decision Tree 在訓練的時候，會逐個嘗試所有 features，並且輪流將 60000 個訓練資料中的同一個 features 作為 threshold 來分割所有訓練資料，找到 Gini impurity 最低的 feature 和 threshold，但因為我們需要找過 780(pixels) * 60000(images) ，並且分別計算他們的 Gini impurity(又是 60000 images)才會得到第一個 split，光是得到第一個分割點就已經非常困難，更何況我們不只有一個分割點。

所以在改進版本中，我們提取出所有的 feature_label pair，並且將他們依照 feature 排序，接下來模擬一個切割器，在陣列的每一個位置分割，我們就可以省去每次計算 Gini impurity 之前，需要對資料按照 threshold 分類的步驟，因為排序好的陣列已經幫我們處理好了這一點，分割線以左都是小於，分割線以右都是大於。此外，我們還每間隔 4 個 pixel 取一次 feature label pair，這樣減少採樣的方法讓原本的 28 * 28 的圖像變成 7 * 7，計算量減少了不少，但同時又仍然保持對數字的識別度。

4 Experiments and Results

4.1 Solution Quality

Model	Accuracy (Test)	F1(Test)	Accuracy (Train)	F1(Train)
Linear Classifier	88.32%	0.881	89.52%	0.894
Decision Tree	82.96%	0.828	86.48%	0.864
KNN	82.03%	0.818	80.80%	0.805

- Linear Classifier 不論在測試資料還是訓練資料有著最高的準確度和 macro F1。
- Decision Tree 表現的比較普通，可能有一點 overfitting，Training dataset 的分數比 test dataset 還要高。
- KNN 的準確率和 F1 都是最低的，大概是因為我們的改進版過於簡化導致數字變得模糊，容易判斷錯誤導致
- Overall Accuracy:

Linear Classifier > Decision Tree > KNN

4.2 Efficiency

Model	Efficiency Notes
KNN (Centroid)	訓練速度快，並且只有儲存 10 個 average vectors
Linear Classifier	儲存 10 張圖片的權重，在預測時很準，但是最久的訓練時間。
Decision Tree	儲存了一整顆樹，有著最快的預測速度，中等的預測準度

- 在記憶體使用的部分，改進過後的 KNN 占用最小的記憶體空間，因為它僅僅是儲存 10 個 average vectors。
- Linear Classifier 的模型因為它儲存權重的方式所以擴展性很好
- Decision Tree 需要較多的記憶體儲存它遞迴呼叫的節點，但是這個是三個方法中預測速度最快的一種

4.3 Running Time (Training + Prediction)

Model	Training Time	Prediction Time (10k test samples)
KNN (Centroid)	997 ms	1402 ms
Linear Classifier	1409630 ms (≈23.5 min)	873 ms
Decision Tree	123722 ms (≈2 min)	7 ms

- 在 Runtime 的部分，三種方法的執行時間差異就非常明顯了，很明顯可以看到 Decision Tree 在預測的部分速度非常快，僅 7ms 就完成了 10000 筆測試資料的預測。得益於 Decision Tree 預測的方法還有最大深度的限制，最多只需要詢問 10 次 if-else，就可以走到葉節點，並且得到預測答案。
- 而 Linear Classifier 竟然花費了 23 分鐘在訓練，不過最後預測的成效不錯
- KNN 在訓練時間方面是最快速的，不過同時也是準確率最低的一種。

4.4 Overall Trade-Off Summary

Model	Best At	Weakness
KNN (Centroid)	訓練速度快、簡單暴力好理解	準確性最差(因為使用平均而不是逐一比較)
Linear	準確性高、擴展性也高	訓練速度很慢
Tree	預測速度最快	訓練花了一點時間，得到中等的準確度

從以上這些特點我們可以發現，想要預測準確率高的模型，基本都需要更久的訓練時間，或是更久的預測時間(例如還沒改進前的 KNN)。想要預測速度快的模型，也需要一些訓練時間。而如果我們花最少的時間訓練，那我們就會得出準確率最低的模型，所以如果想要準確率、速度，那就必須在訓練時間或預測時間這方面有 tradeoff。

5 Discussion and Findings

比較這些 Tradeoffs:

- KNN 訓練時間最快，預測時間不長，但是準確率最低
- Decision Tree 是最平衡的演算法，訓練時間不長，但預測時間非常快，而且也有一定的準確率。
- Linear 訓練時間最久，預測時間短，但準確率最高的演算法

如果速度和不要太高的複雜程度是我們的第一優先考量，那改進過後的 KNN 會是最好的選擇。如果需要準確率，那可以選擇 Linear Classifier，雖然他在訓練部分相當耗時間。而 Decision Tree 就平衡了準確率和訓練速度的問題，我覺得是這之中最均衡的演算法，在還可接受的 trade off 以內，盡可能的拉高準確率和預測速度。

6 Conclusion

在這個報告中，我學到了不同模型訓練和預測的方法，經過這些測試，我覺得 Decision tree 是最好的演算法，因為它有著不錯的訓練時間和預測時間。而且還有優化的空間 (例如使用更多的 pixel 來找到更好的分割點等) 。
因為還有期末報告的壓力，否則我還想繼續探討 Linear 中不同 epoch 和 learning rate 對結果的影響；最原始暴力的 KNN 是否能有著非常高的預測準度；Decision Tree 如果使用更多的 pixels 訓練會不會有不一樣的變化等等。